

GPGPU - Práctico 3: Memoria compartida y programación a nivel de warp

Nery Ezequiel Capó Brazeiro - Grupo 32
Facultad de Ingeniería, Udelar
54132138
Montevideo, Uruguay
nery.ezequiel.capo@fing.edu.uy

Miguel Rodrigues - Grupo 32
Facultad de Ingeniería, Udelar
53986855
Montevideo, Uruguay
miguelrodriguescas1333@gmail.com

I. EJERCICIO 1: SHARED MEMORY

Para las pruebas se emplearon matrices cuadradas de tamaño $2^{14} = 16384$ y bloques de dimensión 32×32 , tal como especifica la consigna.

I-1. Parte 1: Se utilizó memoria compartida mediante tiles de 32×32 . Cada hilo, identificado por sus coordenadas locales (t_x, t_y) dentro del bloque. Cada hilo carga el elemento correspondiente de la matriz original en memoria compartida:

$$\text{tile}[t_y][t_x] = A_{\text{in}}[g_y][g_x]$$

1

Notar que t_x en dicha posición implica accesos coalesced, según la disposición de memoria global en C (row-major).

Luego de sincronizar los hilos del bloque mediante `__syncthreads()`, se recalculan los índices de escritura intercambiando los roles de t_x y t_y :

$$x' = b_x \cdot \text{Tile_dim} + t_y, \quad y' = b_y \cdot \text{Tile_dim} + t_x$$

y se escribe en la matriz resultado leyendo el tile de forma transpuesta:

$$A_{\text{out}}[y'][x'] = \text{tile}[t_x][t_y]$$

Como los accesos más expuestos son en base a t_x , este acceso también es coalesced.

Por tanto, podemos esperar que las transacciones entre shared memory y memoria global sean eficientes en terminos de utilización.

En contraste, una implementación directa sin shared memory presenta accesos no *coalesced* en la escritura (como vimos en el practico anterior).

Si bien los accesos a shared memory son mucho más rápidos que la memoria global, introduce un overhead al tener que sincronizar hilos `__syncthreads()`. E inicialmente al cargar los datos igualmente accedemos a memoria global, pero con la gran ventaja que es coalesced. Por otro lado, la performance puede verse mermada según conflictos de bancos, es decir, accesos de diferentes datos a los mismos

$$^1 g_x = \text{block_idx_x} \cdot \text{block_dim_x} + t_x, \quad g_y = \text{block_idx_y} \cdot \text{block_dim_y} + t_y$$

bancos.

Con todo esto en mente, a continuación se evidencia que la shared memory sigue siendo la opción preferencial, esto gracias a la velocidad de acceso de la shared memory, la misma funciona como caché y permite que escribir previamente y sincronizar hilos sea un coste superficial especialmente cuanto los accesos se definen como coalesced, ya que resulta ampliamente más performante que emplear simplemente memoria global.

Comparación shared vs global, tamaños $N = 2^{14} = 16384$:

Grid Size	Block Size	Tiempo Promedio (ms)	Desv. Est. (ms)
(512, 512)	(32, 32)	7.6801	1.2502

Cuadro I: Resultados de ejecución del kernel (10 runs) shared memory

I-1a. A nivel de experimentacion:: Tener en cuenta que la primera ejecución de las 10 en promedio siempre se desvía bastante, el tiempo lo medimos mediante `for(i = 0; i < 10; i++) < kernel >` y la herramienta nsys. Pero probamos previamente medir con cudaEvents, 11 veces eliminando la primera ejecución y el promedio arrojado fue: 5.596560 (ms) y desv std: 0.002585. Esto se evidencia en gran medida por la desviación estandar, implica que la primera de las 10 ejecuciones es muy poco performante comparada con las demás.

Grid Size	Block Size	Tiempo Promedio (ms)	Desv. Est. (ms)
(512, 512)	(32,32)	11.8080	-
(4096, 256)	(4,64)	5.8752	-

Cuadro II: Resultados de ejecución del kernel (5 runs) global memory

Como se puede observar el rendimiento mejora notablemente referido a la versión de acceso global con igual tamaño de bloque. Por otro lado, su mejora no es tan evidente respecto al tamaño de bloque alterado (4,64) que logra mitigar el efecto de accesos no coalesced.

I-2. *parte 2::* Dicha parte impone aumentar la dimensión del tile a nivel de columna, todo esto para evitar conflictos de banco (bank conflicts) en shared memory .Es decir establecer un padding:

$$\text{tile}[32][32 + 1]$$

Grid Size	Block Size	Tiempo Promedio (ms)	Desv. Estándar (ms)
(512, 512)	(32, 33)	5.2807	0.0037

Cuadro III: Resultados de ejecución del kernel (10 runs)

Se nota una gran mejoría, alrededor de 45 % en tiempos de ejecución padding vs no padding, además las operaciones con padding son muchísimo más estables en dichas métricas (menor desviación est.)

Teóricamente el empleo de padding es recomendado para cualquier implementación, esto para evitar conflictos de bancos de la shared memory. Sabemos que la shared memory opera a nivel de bloque y que se cargan 32 bancos [0..31], dichos bancos permiten en un ciclo acceder a 4 Bytes de datos (tipo float 4B), por tanto podríamos acceder a 32 elementos por acceso a sharedm memory a nivel de bloque. Las lecturas de Bytes de igual dato no supone conflicto, ahora las escrituras y lecturas de diferentes datos para el mismo banco si.

El banco destinado a una dirección de memoria (nivel 2D) se asume que se calcula: $\text{banco} = (\text{fila} \cdot \text{ancho_fila} + \text{columna}) \% \text{nro_bancos}^3$ (Los bancos de memoria compartida están organizados de manera que palabras consecutivas de 32 bits se asignan a bancos consecutivos, no especifica puntualmente la arquitectura Turing como carga direcciones) [1]. Especialmente al transponer la matriz, tomamos una fila y la colocamos como columna. Tendríamos que cargar filas y columnas en bancos diferentes para maximizar el acceso de los bancos.

Tenemos que a nivel de lecturas (por columnas de misma fila) en TILES[32][32] obtendríamos accesos eficientes

Thread	Acceso	Banco
0	$\text{tile}[ty][0]$	$(ty \cdot 32 + 0) \% 32$
1	$\text{tile}[ty][1]$	$(ty \cdot 32 + 1) \% 32$
2	$\text{tile}[ty][2]$	$(ty \cdot 32 + 2) \% 32$
$\vdots$	$\vdots$	$\vdots$
31	$\text{tile}[ty][31]$	$(ty \cdot 32 + 31) \% 32$

Cuadro IV: Acceso a shared memory, por columnas, sin padding (tile[32][32], ty constante.), Sin conflictos

²No se quisieron alterar las métricas del práctico pasado, los promedios obtenidos son en base a 5 ejecuciones, y suelen fluctuar más respecto a ejecuciones (en torno a 0,5 – 1 ms)

³Acceso lineal de los elementos (a nivel de C) módulo el nro de bancos para asignación

Pero al tomar la traspuesta (Por filas de misma columna) esto cambia y vemos múltiples conflictos: Los conflictos

Thread	Acceso	Banco
0	$\text{tile}[0][ty]$	$(0 \cdot 32 + ty) \% 32 = ty$
1	$\text{tile}[1][ty]$	$(1 \cdot 32 + ty) \% 32 = ty$
2	$\text{tile}[2][ty]$	$(2 \cdot 32 + ty) \% 32 = ty$
$\vdots$	$\vdots$	$\vdots$
31	$\text{tile}[31][ty]$	$(31 \cdot 32 + ty) \% 32 = ty$

Cuadro V: Acceso a shared memory, por filas, sin padding (tile[32][32], ty constante.), Con conflictos

se dan porque todas las direcciones de filas de una misma columna apuntan al mismo banco, definido por ty.

Ahora, teniendo en cuenta la definición de shared memory con padding, para el caso TILE[32][33] (Por columnas de misma fila) se evidencia que no ocurren conflictos:

Thread	Acceso	Banco
0	$\text{tile}[ty][0]$	$(ty \cdot 33 + 0) \% 32$
1	$\text{tile}[ty][1]$	$(ty \cdot 33 + 1) \% 32$
2	$\text{tile}[ty][2]$	$(ty \cdot 33 + 2) \% 32$
$\vdots$	$\vdots$	$\vdots$
31	$\text{tile}[ty][31]$	$(ty \cdot 33 + 31) \% 32$

Cuadro VI: Acceso a shared memory, por filas, con padding (tile[32][33], ty constante.), Sin conflictos

Para el caso, por filas de misma columna, pero con padding (tile [32][33])

Thread	Acceso	Banco
0	$\text{tile}[0][ty]$	$(0 \cdot 33 + ty) \% 32 = ty$
1	$\text{tile}[1][ty]$	$(1 \cdot 33 + ty) \% 32 = (1 + ty) \% 32$
2	$\text{tile}[2][ty]$	$(2 \cdot 33 + ty) \% 32 = (2 + ty) \% 32$
$\vdots$	$\vdots$	$\vdots$
31	$\text{tile}[31][ty]$	$(31 \cdot 33 + ty) \% 32 = (31 + ty) \% 32$

Cuadro VII: Acceso a shared memory, por columnas, con padding (tile[32][33]), ty constante.), Sin conflictos

Se puede evidenciar que los accesos transpuestos no tienen los conflictos que imponía el esquema sin padding, y como es de esperarse mejora la performance. También permite que las ejecuciones sean muchísimo más estable, (desviación estándar menor). Tener en cuenta que la inclusión de padding, implica definir espacio en shared memory que no se usa, solo sirve para disponer direcciones de bancos diferentes y reducir conflictos. Además, como observación es muy probable que el overhead de la versión sin padding se reduzca luego de la primera ejecución por empleo de cachés.

II. EJERCICIO 2: WARP SHUFFLE

Se busca implementar una función que realice: *Para arreglos de tamaño múltiplo de 32, por cada segmento de*

32 enteros consecutivos se debe hacer lo siguiente: Los elementos negativos del segmento deben ser reemplazados por la suma de estos más el máximo de los elementos positivos del segmento.

Esto se logró definiendo un arreglo de tamaños (arbitrarios, uno de gran tamaño y otro de menor) $N = 2^{14}, 2^{28}$ y disponiendo un valor(entero) al azar entre -100 y 100. Para ambas partes definimos el tamaño de bloque igual a 32, esto facilita el código, ya que justamente los segmentos son de 32 elementos, por tanto la organización de hilos (a nivel de bloque) corresponden a un segmento entero del arreglo.

II-A. Parte 1

Solo se permite emplear shared memory, empleamos dos versiones una versión alternativa lineal y otra que aprovecha la logica del warp reduction, pero a nivel de shared memory. Para esto definimos Tile de 34 elementos [0..33], los primeros [0..31] contienen los elementos del arreglo.

Luego, solamente el hilo 0 (a nivel de bloque), recorre los 31 elementos restantes, accediendo a los tiles correspondientes de shared memory, logrando definir el maximo y la suma de los elementos negativos localmente (a nivel de registro), luego escribe los resultados en tile[32] y tile[33]. Finalmente por nivel de cada hilo, si el elemento es negativo, se sobrescribe su valor con tile[33] + tile[32], si no se deja con su valor anterior.

Esta es una implementación sumamente sencilla, que a nivel de recorrida impone una secuencialidad al primer hilo de cada bloque, considerando que son 31 elementos, las recorridas deben ser relativamente eficientes, ya que accede a shared memory, por tanto para el caso actual, consideramos es una implementación acorde.

Los tiempos de ejecución para los arreglos con tamaño de bloque 32 fue:

Grid Size	Tam. arreglo	Tiempo Promedio (ms)	Desv. Est. (ms)
512	2^{14}	0.0021696	0.0001832
8388608	2^{28}	7.2086	0.90308

Cuadro VIII: Resultados de ejecución del kernel (10 runs) función para arreglo, shared memory, alternativa acceso lineal

Para la versión con mayor utilización de hilos, separados dos espacios de shared memory. tiles_neg[32] y tiles_max[32], de tal manera de recorrer los negativos y positivos y depositar en el tiles_neg[0] el negativo de la suma y tiles_max[0] el maximo correspondiente, con una logica análoga a la reducción de warp. De tal manera que estaríamos aprovechando mejor los hilos a nivel de trabajo.

Los tiempos de ejecución para los arreglos con tamaño de bloque 32 fue:

Se puede observar que al aumentar el tamaño del arreglo, la ejecución a nivel de shared memory sufre tiempos de cómputo

Grid Size	Tamaño arreglo	Tiempo Promedio (ms)	Desv. Est. (ms)
512	2^{14}	0.0026432	0.0001805
8388608	2^{28}	11.9793	1.7921

Cuadro IX: Resultados de ejecución del kernel (10 runs) función para arreglo, shared memory

mayores, siendo la versión alternativa lineal mucho mejor. Además la alternativa lineal tiene menor desviación estándar, intuitivamente porque el acceso a los tiels está serializado, no impone sobrecargas por sincronizaciones ni potenciales conflictos

II-B. Parte 2

Para esta parte se permite emplear primitiva a nivel de warp, puntualmente empleamos `__ballot_sync`, para obtener los valores negativos del arreglo (según su máscara), `__shfl_down_sync` para reducción de warp y obtener tanto máximo(max) del arreglo y suma de los negativos(negSum), y finalmente `__shfl_sync` para hacer broadcast de los valores negSum y max, solamente para los hilos que están con la mascara de valores negativos.

Para la mascara del `__shfl_down_sync`. Empleamos todos los lanes del warp, es decir todos los hilos del bloque, ya que la mascara solo define la sincronización a nivel de la ejecución del warp, por tanto incluir otra mascara implica que los valores fuera de esta sean indeterminados.

Buscamos estrategias para segregar valores positivos y negativos con el fin de paralelizar el acceso. Sin embargo, no obtuvimos conclusiones definitivas. Experimentalmente, pudimos aplicar máscaras diferenciadas para negativos y positivos en las operaciones de suma de negativos y máximo (en sus respectivas reducciones):

```
__shfl_down_sync(negMask, val, offset)
__shfl_down_sync(posMask, val, offset)
```

Y dicha implementación logró funcionar correctamente, sin embargo, de acuerdo con la bibliografía, este enfoque no debería emplearse, se recomienda utilizar valores neutros para garantizar que la propagación funcione correctamente tanto en los resultados como en la convergencia.

También probamos la alternativa de no ejecutar `__shfl_down_sync` en los hilos objetivos donde no era necesario, mediante condiciones explícitas(bitwise):

```
if(lane_id+offset < 32&&(1u <= (lane_id+offset))&negMask){
if(lane_id+offset < 32&&(1u <= (lane_id+offset))&posMask){
```

Dicha implementación y alternativas no pudieron ser ejecutadas, dieron diversos errores, los cuales no hallamos solución.

Los tiempos de ejecución para los arreglos con tamaño de bloque 32 fue:

Grid Size	Tam arreglo	Tiempo Promedio (ms)	Desv. Est. (ms)
512	2^{14}	0.0020835	0.000178
8388608	2^{28}	6.3735	0.002923

Cuadro X: Resultados de ejecución del kernel (10 runs) función para arreglo, warp shuffle

Es notable la mejora de performance a nivel de warps, alrededor de un 88 % con la versión análoga de shared memory, y un 13 % con la versión alternativa lineal (para el tamaño de arreglo 2^{28}).

Evidentemente la mejora de performance se hace más notable al emplear tamaños de arreglo mayores, esto intuitivamente podría resultar por saturar la shared memory lo cual la degrada según cuán grande es el arreglo. A diferencia de la versión a nivel de warp las demoras estarían mas bien acotadas por las transferencias a nivel de registros y los cálculos dentro de la GPU.

Teóricamente al ser segmentos pequeños de 32 elementos, warp shuffle resulta ser la mejor solución, ya que cada warp opera a nivel de 32 hilos y el alcance de primitivas a nivel de warp sería más eficiente por el alcance del problema.

Porque la implicancia de shared memory se define a nivel de bloques, es decir, su alcance permite operar a nivel mayor que warp, y su overhead suele ser más degradante que el que puede disponer las primitivas a nivel de warp. Y los resultados fueron acordes a las ejecuciones.

III. CONCLUSIONES

En este trabajo se evidenció la utilidad de distintas estrategias de optimización para uso de memoria, así como sus implicancias a nivel de arquitectura.

En particular, el uso de *shared memory* demostró ser clave para mejorar los tiempos de acceso respecto a memoria global. Se analizó el impacto de los *bank conflicts*, observándose que pueden degradar significativamente la performance si no se gestionan adecuadamente. La incorporación de *padding* permitió eliminar dichos conflictos, validando en la práctica los conceptos teóricos. Asimismo, se reafirmó que la *shared memory* opera a nivel de bloque, actúa como una caché y requiere típicamente de la sincronización a nivel de bloque, mediante `__syncthreads()`. Por otro lado, se demostró que los accesos coalesced resultan determinantes para alcanzar una performance ideal.

Se exploró el uso de primitivas a nivel de warp, evidenciando su utilidad como alternativa a *shared memory* en problemas cuyo alcance se ajusta al tamaño de un warp. Estas primitivas presentan menor latencia al evitar sincronizaciones a nivel de bloque y accesos a memoria compartida. Sin embargo, su alcance está limitado a la ejecución dentro de un warp, lo que restringe su aplicabilidad a ciertos patrones de cómputo.

También se pudo comprender el funcionamiento de varias a nivel de warp, donde la utilidad de la máscara se restringe a la

sincronización y convergencia de ejecución a nivel de lanes del warp indetificados, se falló en hallar una versión paralelizable a nivel de ejecución de primitivas con `__ballot_sync` y las máscaras obtenidas, la ejecución fue serializada, ya que así lo impuso la bibliografía que hallamos.

En síntesis, el contenido teórico pudo ser aplicado de manera práctica, permitiendo su evaluación en un contexto real. Comparando diferentes implementaciones y su performance relativa.

También, se logró emplear la herramienta Nsys para la obtención de métricas.

IV. ANEXO

Para el Ejercicio 1 se definieron tres casos de prueba: en primer lugar, la verificación funcional de la transposición de la matriz. luego, la medición de tiempos de ejecución sin padding. Y finalmente, la evaluación con padding.

Para el Ejercicio 2 (Parte 1), se validó inicialmente el correcto funcionamiento de la función sobre un arreglo de tamaño 64 (dos segmentos), seguido de la medición de rendimiento, con arreglo de tamaño 2^{28} .

Para el Ejercicio 2 (Parte 2), se siguió un procedimiento análogo: verificación del funcionamiento arreglo de tamaño 64 y posterior evaluación de performance, con arreglo de tamaño 2^{28} .

Equipo empleado, Gráfica Nvidia GTX 2080Ti.

REFERENCIAS

- [1] Mark Harris, *Using Shared Memory in CUDA C/C++*, NVIDIA Technical Blog, 28 de enero de 2013. Disponible en: <https://developer.nvidia.com/blog/using-shared-memory-cuda-cc/>
- [2] Facultad de Ingeniería - Universidad de la República, *Clase 6 - Programación CUDA 2*, Disponible en: https://eva.fing.edu.uy/pluginfile.php/98384/mod_label/intro/Clase%206%20-%20ProgramacionCUDA2_2026.pdf, Accedido: abril 2026.